# CODESIGN

DBIBIH Oussama

# TP VHDL CODESIGN 2026

| Groupe | Validation | Tuteur |
|---|---|---|
| **DBIBIH Oussama** | **07.03.2026**<br>Signature récupérable<br>X VALIDE<br>DBIBIH Oussama<br>Apprenti Ingenieur<br>Signé par : f40a5d3a-c8fb-49bd-90dc-17d5deacb45b | Thierry PERISSE |
| **CASTELO Thomas** | | |

# TABLE DES MATIERES


**I. INTRODUCTION** ..... 3

**II. PREMIER PROJET QUARTUS** ..... 4

**1. Prise en main** ..... 6

***2. Exercice 1*** ..... 6

***3. Exercice 2*** ..... 7

***4. Exercice 3*** ..... 8

***5. Exercice 4*** ..... 9

***6. Exercice 5*** ..... 10

***7. Exercice 6*** ..... 11

**III. CONCEPTION DES SOC/SOPC** ..... 12

***1. Partie 1 : développement du matériel*** ..... 12

***2. Partie 2 : développement logiciel*** ..... 12

***3. Partie 3 : intégration d'un composant propriétaire*** ..... 13

**IV. SPECIFICATIONS CIRCUIT ACQUISITION VITESSE VENT (ANEMOMETRE)** ..... 14

***1. Analyse fonctionnelle de la machine à états*** ..... 14

***2. Vérification de la simulation :*** ..... 15

**V. CONCLUSION** ..... 17


# TABLE DES FIGURES


Figure 1 - Fenêtre VS code ..... 4
Figure 2 - Fenêtre VS Code & README ..... 5
Figure 3 - ModelSim pour la prise en main ..... 6
Figure 4 – TB une horloge de période 20 ns ..... 6
Figure 5 – TB une horloge de 20 ns (50 MHz) et un compteur ..... 7
Figure 6 - TB le signal 50 MHz en un signal lent (1 Hz) ..... 8
Figure 7 - ModelSim pour l'exo 3 avec reset ..... 8
Figure 8 - TB la machine à états (MAE) ..... 9
Figure 9 – TB les états internes `ep` (état présent) et `es` (état suivant) ..... 9
Figure 10 – TB le compteur + Bouton go ..... 10
Figure 11 – TB la temporisation d'anti-rebond ..... 11
Figure 12 - Photo qui visualise le PWM sous l'oscilloscope ..... 13
Figure 13 - Illustration de la machine a état de l'anémomètre ..... 14
Figure 14 - Simulation TB pour CLK_periode 8 ns et 900ms de simulation ..... 15
Figure 15 - Simulation TB pour la mae de l'anémomètre ..... 16

# I. INTRODUCTION

Dans le cadre du module ***Codesign***, ce projet a pour objectif d'explorer la conception conjointe matériel / logiciel sur FPGA à travers l'utilisation de VHDL, de l'environnement **Quartus Prime 18**, de **ModelSim** et de la carte **DE0-Nano**.

L'approche codesign consiste à répartir les fonctionnalités entre la partie matérielle (logique programmable) et la partie logicielle (processeur embarqué).

Ce travail débute par une prise en main progressive des outils de développement FPGA, puis évolue vers la conception de systèmes numériques de plus en plus complexes :

- Implémentation de compteurs synchrones et asynchrones en VHDL
- Conception et validation de machines à états finis (MAE)
- Gestion de contraintes temporelles (division d'horloge, anti-rebond 20 ms)
- Simulation fonctionnelle sous ModelSim
- Validation expérimentale sur carte DE0-Nano

Les premiers exercices permettent de maîtriser les fondamentaux du design numérique : description comportementale, synchronisation sur horloge 50 MHz, gestion du reset et validation par testbench.

Par la suite, le projet s'oriente vers une architecture plus avancée avec la conception d'un SOPC (System On Programmable Chip) à l'aide de Platform Designer, intégrant le processeur softcore NIOS II et des périphériques personnalisés. Cette étape illustre :

- Conception matérielle dans Quartus Prime
- Simulation sous ModelSim
- Développement logiciel embarqué
- Intégration de composants propriétaires (ex : PWM)

Ce travail constitue ainsi une application concrète des principes fondamentaux de l'ingénierie des systèmes embarqués et du design numérique sur FPGA.

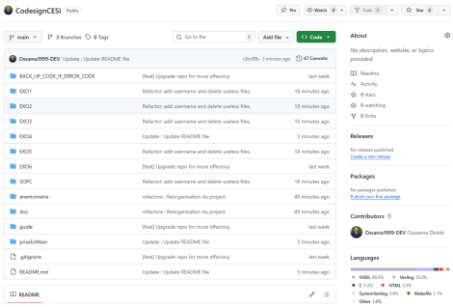
## II. PREMIER PROJET QUARTUS

Pour organiser le travail et assurer une bonne gestion du code, j'ai choisi **Git** comme gestionnaire de versions et **GitHub** comme dépôt distant.

Cela m'a permis de maîtriser le déroulement du projet ainsi que le suivi des modifications effectuées au cours des séances de TP.

Vous trouverez mon dépôt ***GitHub*** à l'adresse suivante :

*Ossama1999-DEV/CodesignCESI* : *https://github.com/Ossama1999-DEV/CodesignCESI*



L'ensemble des dossiers ***EXO1 à EXO6***, ***priseEnMain***, ***SOPC*** et ***anemometre*** contient un fichier ***README.md*** détaillant les objectifs, les explications et les réalisations effectuées durant les séances de TP.

Pour consulter ces fichiers README :

- Ouvrir chaque dossier dans ***VS Code*** ou via le dépôt ***GitHub***.
- Sous VS Code, utiliser le raccourci Ctrl + K puis V afin d'afficher l'aperçu ***Markdown***.

Ces documents permettent de comprendre précisément la démarche, les choix techniques et les validations réalisées pour chaque exercice.

*Figure 1 - Fenêtre VS code*

Pour visualiser le ***README.md*** :

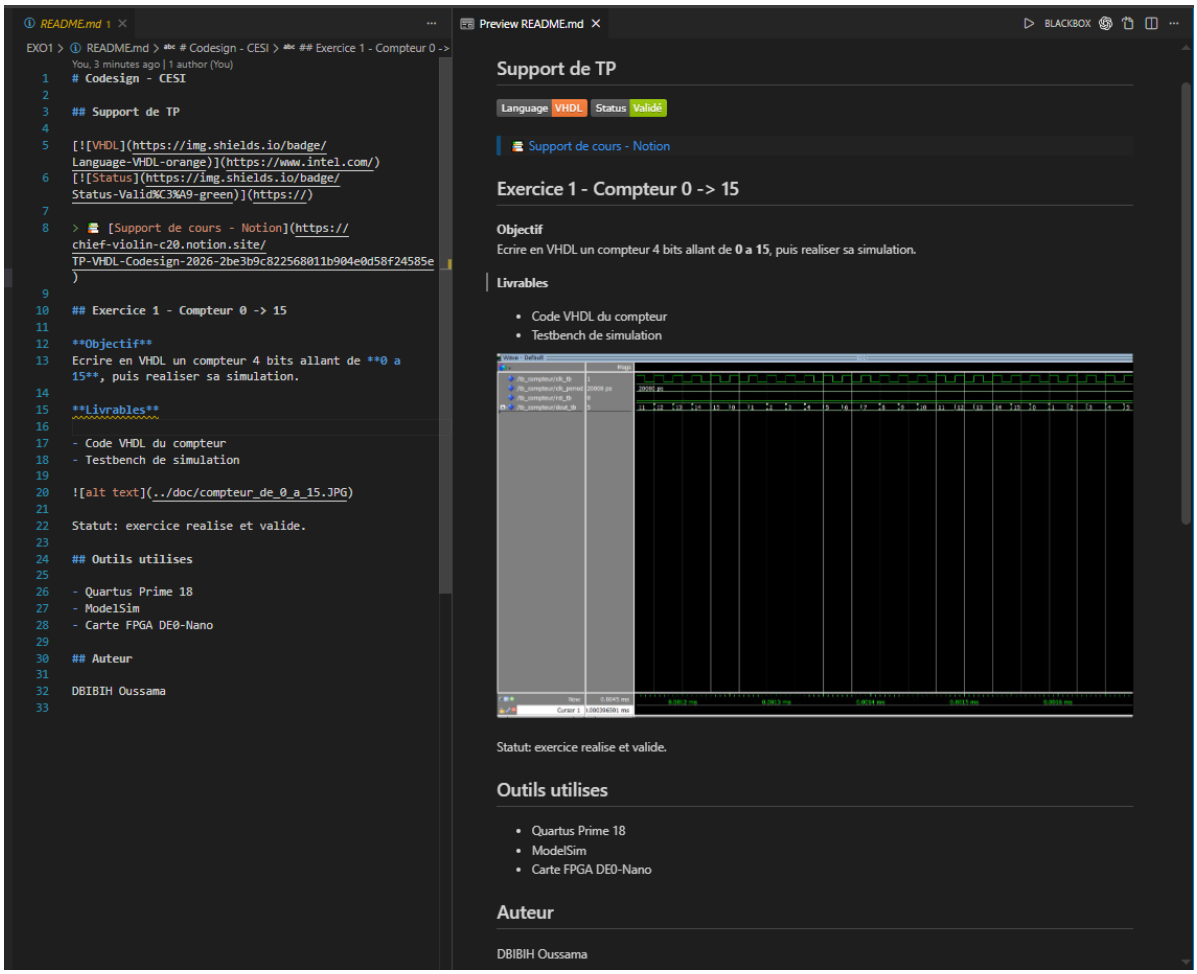



*Figure 2 - Fenêtre VS Code & README*

## 1. Prise en main

Ce module VHDL réalise un **diviseur de fréquence** permettant de transformer l'horloge de la carte **DE0-Nano** (50 MHz) en un signal lent d'environ **1 Hz** afin de faire clignoter une LED toutes les secondes.

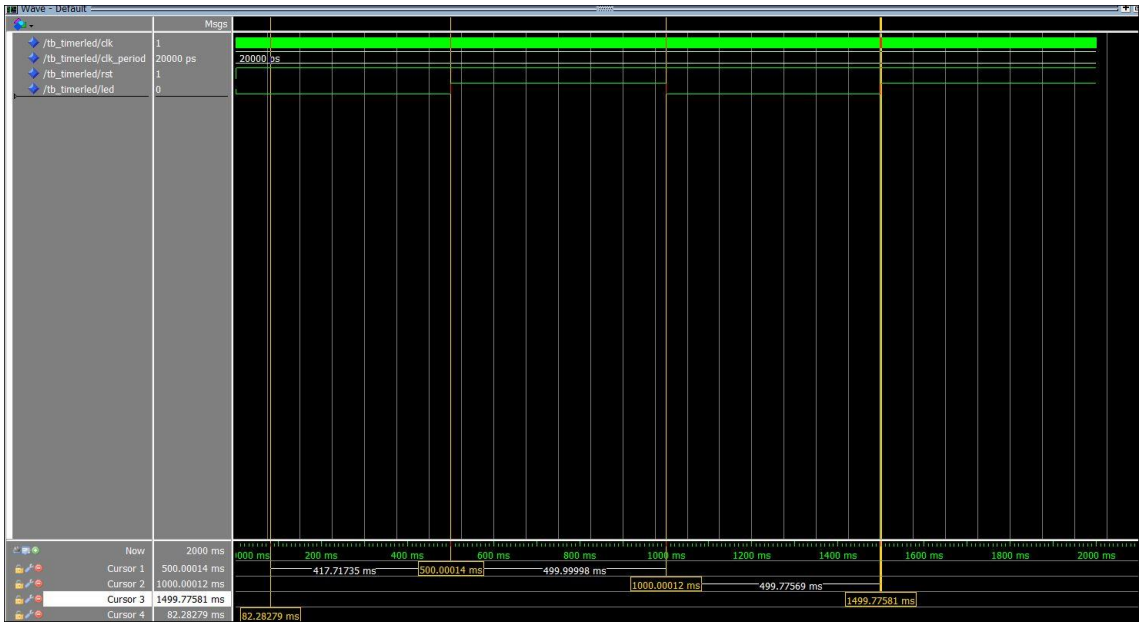



*Figure 3 - ModelSim pour la prise en main*

## 2. *Exercice 1*

Ce module est un **compteur 4 bits**.

Il compte de **0 à 15**, puis revient à 0 et recommence.

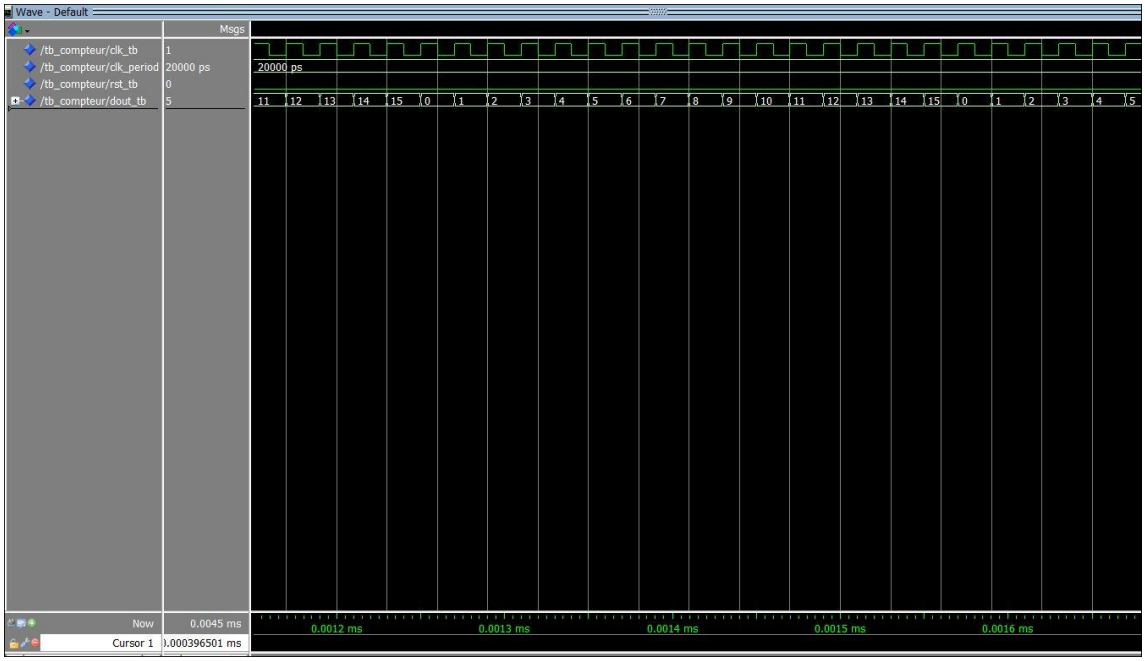



*Figure 4 – TB une horloge de période 20 ns*

La simulation montre une horloge de période 20 ns, correspondant à une fréquence de 50 MHz.

On observe que la sortie bascule périodiquement après un nombre déterminé de cycles d'horloge, validant le bon fonctionnement du compteur.

Le comportement est conforme au cahier des charges : comptage correct et remise à zéro automatique.

### 3. *Exercice 2*

Objectif : Réaliser un compteur synchrone 4 bits qui compte de **0 à 9**, puis revient à 0

Fonctionnement : À chaque front montant de l'horloge, le compteur s'incrémente ; lorsqu'il atteint 9 (1001), il se remet automatiquement à 0.

Le signal rst permet une remise à zéro synchrone du compteur.

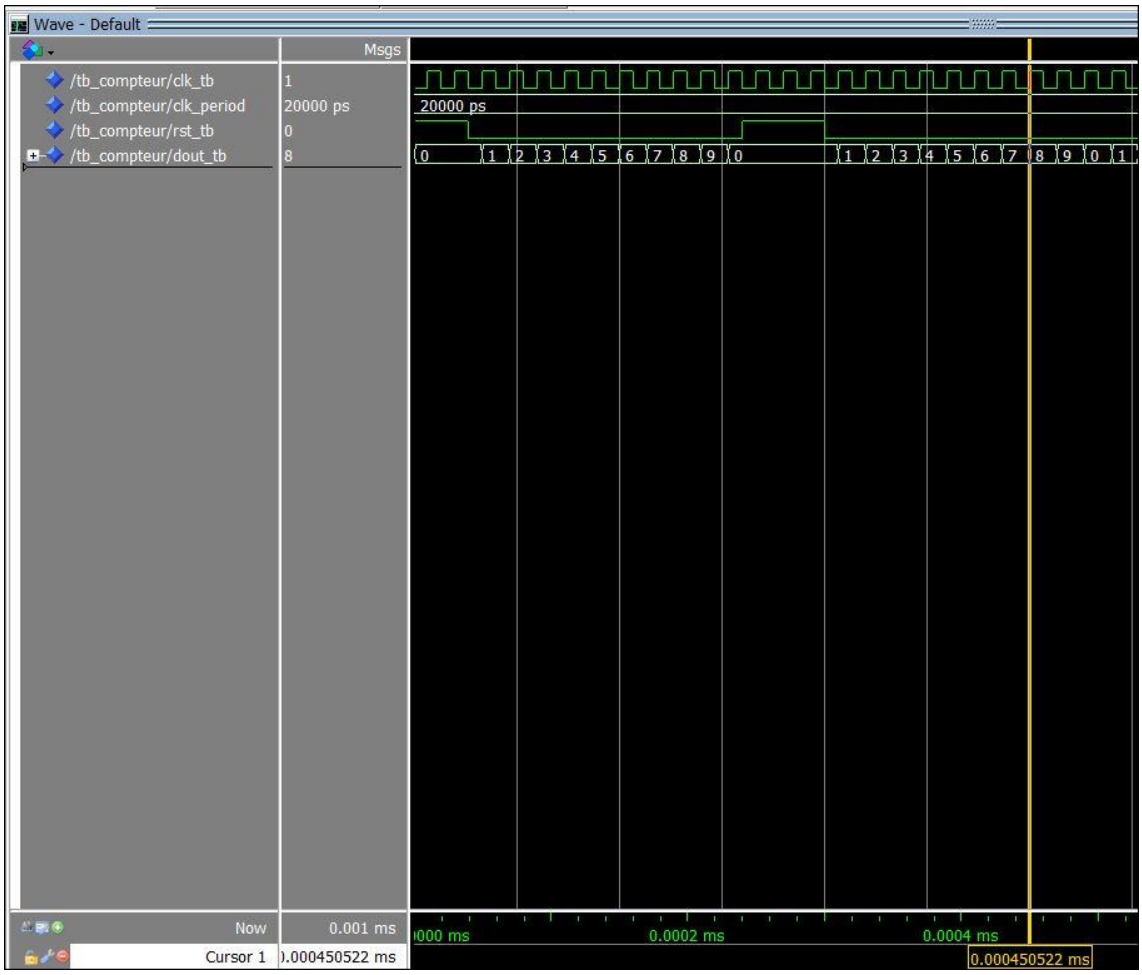



*Figure 5 – TB une horloge de 20 ns (50 MHz) et un compteur*

La simulation montre une horloge de 20 ns (50 MHz) et un compteur qui s'incrémente à chaque front montant.

La sortie ***dout*** compte de 0 à 9 puis revient à 0, conformément au compteur décimal demandé.

Le signal ***rst*** remet correctement le compteur à zéro, validant le bon fonctionnement de l'Exercice 2.

## 4. *Exercice 3*

Objectif : Réaliser un compteur synchrone de 0 à 9 avec une incrémentation toutes les 1 seconde à partir de l'horloge 50 MHz de la DE0-Nano.

Fonctionnement : Un diviseur d'horloge transforme les 50 MHz en un signal lent (1 Hz), qui pilote ensuite le compteur.

Le compteur s'incrémente chaque seconde, revient à 0 après 9, et peut être remis à zéro avec ***rst***.

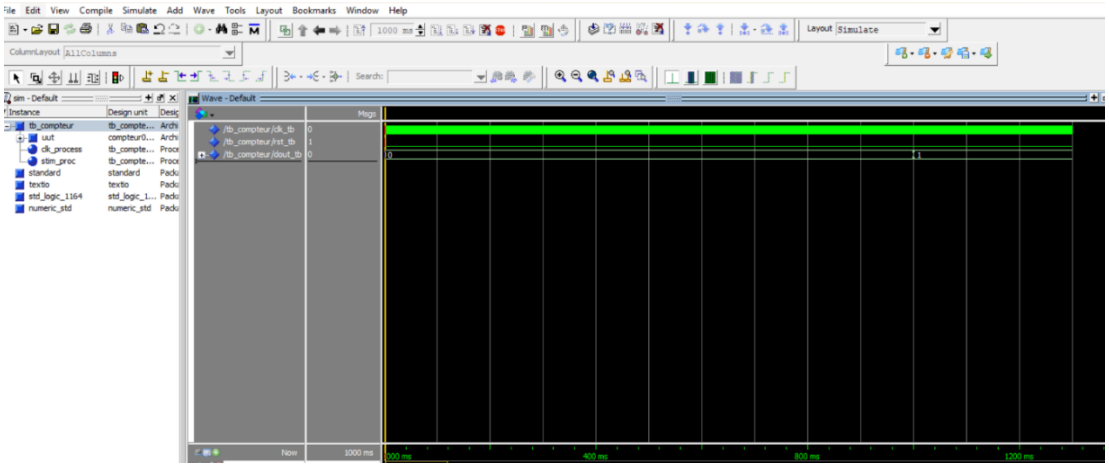


*Figure 6 - TB le signal 50 MHz en un signal lent (1 Hz)*

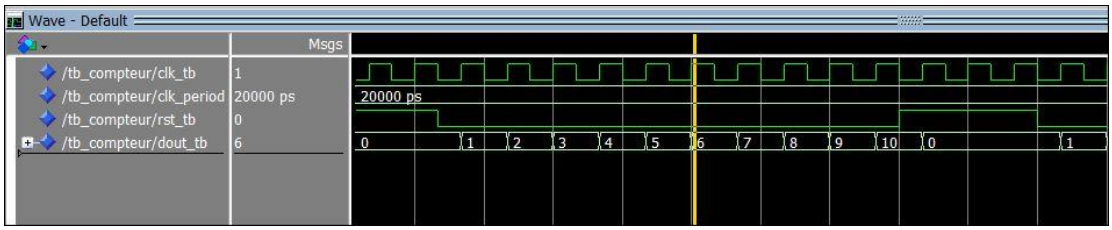



*Figure 7 - ModelSim pour l'exo 3 avec reset*

La simulation montre que le diviseur d'horloge transforme le signal 50 MHz en un signal lent (1 Hz).
Le compteur s'incrémente uniquement à chaque impulsion du signal divisé, soit toutes les 1 seconde.
La sortie `dout` compte correctement de 0 à 9 puis revient à 0.

## 5. *Exercice 4*

Objectif : Réaliser une machine à états finis (MAE) de type Moore avec deux états (etat1, etat2).

Fonctionnement : Selon les entrées in1 et in2, la machine change d'état à chaque front montant d'horloge ; le reset force l'état initial etat1.

La sortie out1 dépend uniquement de l'état présent (0 en etat1, 1 en etat2).

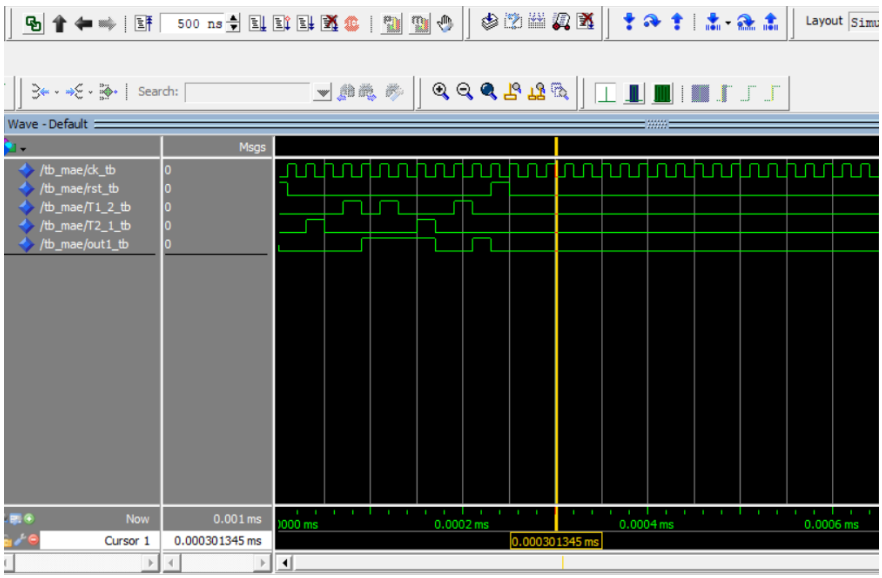



*Figure 8 - TB la machine à états (MAE)*

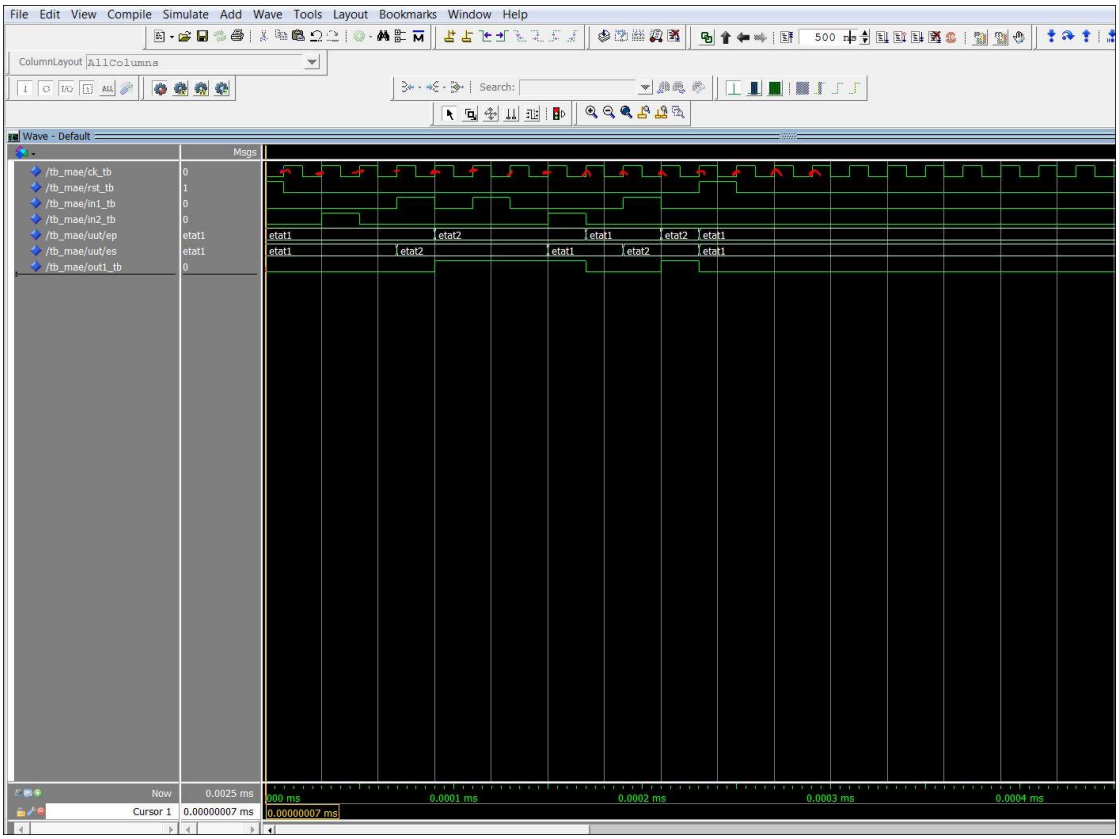



*Figure 9 – TB les états internes `ep` (état présent) et `es` (état suivant)*

La simulation montre le fonctionnement de la **machine à états (MAE)** avec deux états.
Selon les entrées `T1_2` et `T2_1`, la machine change d'état sur front montant d'horloge, tandis que le reset force le retour à l'état initial.
La sortie `out1` dépend uniquement de l'état présent (machine de Moore), ce qui valide le comportement attendu de l'Exercice 4.

La simulation affiche clairement les états internes `ep` (état présent) et `es` (état suivant), montrant les transitions entre `etat1` et `etat2`.
Les changements d'état se produisent uniquement sur front montant d'horloge, et le reset force le retour à `etat1`.
La sortie `out1` suit l'état présent

## 6. *Exercice 5*

Réaliser un compteur 4 bits qui s'incrémente **une seule fois par appui sur le bouton** `go`, avec un **anti-rebond de 20 ms** pour éviter les fausses détections.

Le bouton passe par une machine à états (MAE) qui :

a. Vérifie que l'appui dure au moins 20 ms (anti-rebond).
b. Génère une impulsion d'un seul cycle (go_compte).
c. Le compteur s'incrémente uniquement sur cette impulsion.

Même si on reste appuyé longtemps → une seule incrémentation.

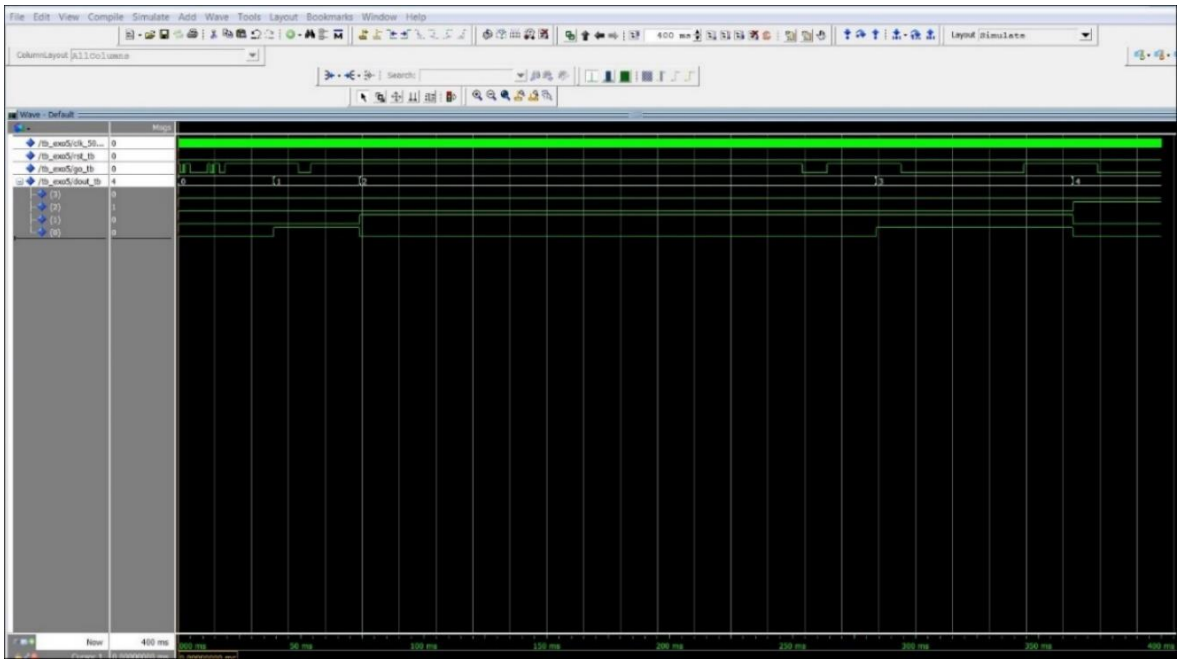


*Figure 10 – TB le compteur + Bouton go*

La simulation montre que le compteur s'incrémente uniquement après un appui valide sur le bouton `go`.
Les impulsions courtes (rebonds) sont ignorées grâce à la temporisation de 20 ms, et une seule impulsion `go_compte` est générée par appui.
La sortie `dout` augmente d'une unité par appui

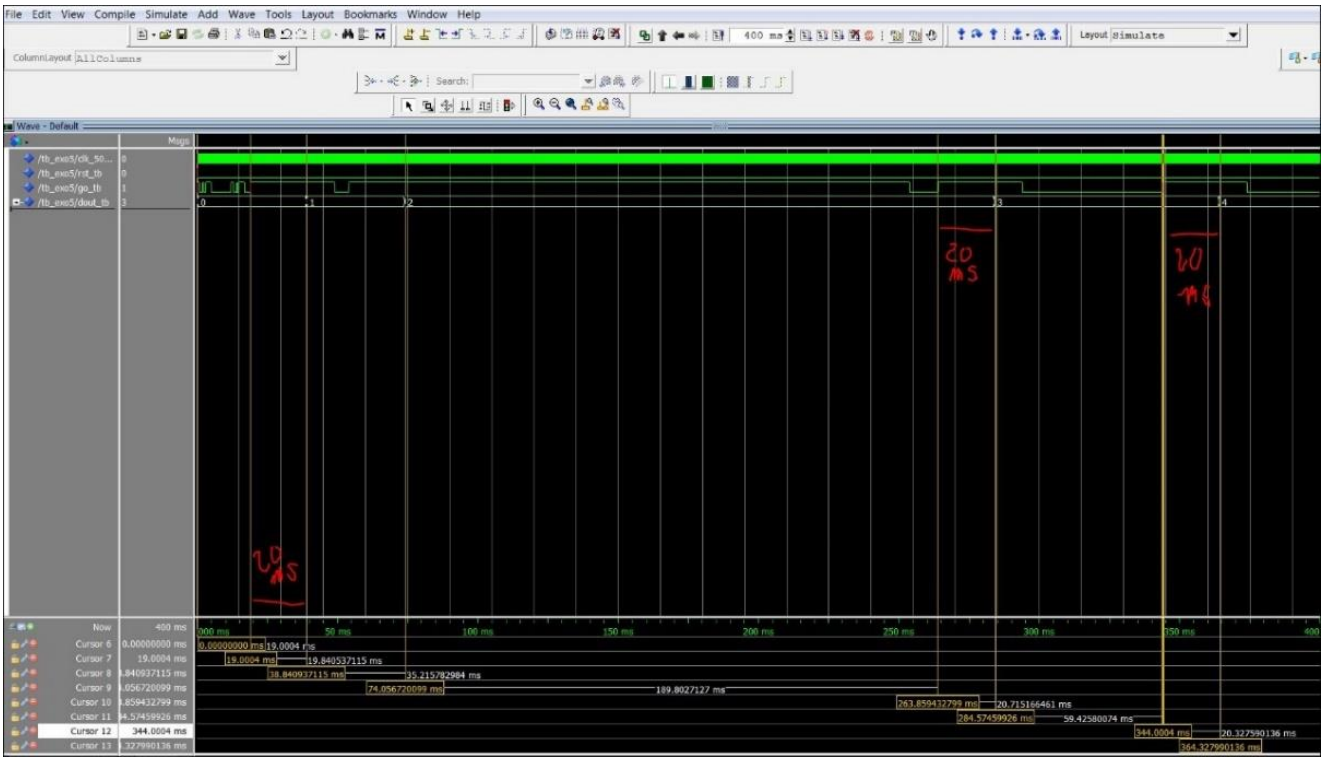



*Figure 11 – TB la temporisation d'anti-rebond*

La figure met en évidence la **temporisation d'anti-rebond de 20 ms** : après l'appui sur `go`, la MAE attend ~20 ms avant de valider l'action.
Une fois ce délai atteint, elle génère une **impulsion unique `go_compte` (1 cycle d'horloge)**, même si le bouton reste appuyé.
On observe alors que `dout` **s'incrémente une seule fois par appui**, et que les impulsions plus courtes que 20 ms sont ignorées (rebonds).

EXO5/exo5_top.vhd

EXO5/mae_go_20ms.vhd et EXO5/TB_exo5.vhdl

## 7. *Exercice 6*

Concevoir et tester sur carte **DE0-Nano** un **SOPC "sopc_compteur"** (microcontrôleur dans le FPGA) avec un processeur **Nios II**, une mémoire on-chip, 2 entrées boutons et 8 sorties LEDs.

Avec **Platform Designer**, on assemble le matériel (CPU + RAM + PIO boutons/LEDs + JTAG UART), on génère le système puis on compile et programme le FPGA sous Quartus. Ensuite, on développe en C sous Nios II IDE un programme qui **incrémente une variable et l'affiche sur les LEDs**, avec les boutons pour **reset** et **pause/blocage** du comptage.

Affichage de hello world dans la terminale de Nios 2 et clignotant des LED.

# III. CONCEPTION DES SOC/SOPC

## 1. *Partie 1 : développement du matériel*

L'objectif de cette partie était de concevoir un système SOPC (System On Programmable Chip) nommé sopc_compteur à l'aide de Quartus Prime 18 et Platform Designer, puis de l'implémenter sur la carte DE0-Nano (Cyclone IV).

- Le système matériel comprend :
- Un processeur softcore NIOS II (version économique)
- Une mémoire RAM On-Chip (20 Ko)
- Un PIO 2 bits en entrée (boutons)
- Un PIO 8 bits en sortie (LEDs)
- Un module JTAG UART pour la communication avec le PC
- Un composant System ID pour la sécurité

Les étapes réalisées :

- Création du système dans Platform Designer
- Attribution automatique des adresses (Assign Base Address)
- Génération des fichiers HDL *(.vhd / .qip)*
- Intégration dans Quartus
- Affectation des broches via Pin Planner
- Compilation et génération du bitstream *.sof*
- Programmation du FPGA

À ce stade, le microcontrôleur personnalisé était entièrement fonctionnel sur la carte.

## 2. *Partie 2 : développement logiciel*

Cette partie consiste à développer un programme en langage C sous Nios II IDE, exécuté sur le processeur NIOS II intégré au FPGA.

Les étapes réalisées :

- Création du projet logiciel à partir d'un template
- Génération du BSP (Board Support Package)
- Compilation du programme
- Téléchargement et exécution sur la carte

Le programme développé permet :

- L'incrémentation d'une variable
- L'affichage de cette valeur sur les 8 LEDs
- L'utilisation des boutons pour :
    - Remise à zéro
    - Blocage du comptage

### 3. *Partie 3 : intégration d'un composant propriétaire*

Dans cette partie, un composant matériel spécifique (générateur PWM) a été conçu et intégré au **/SOPC**.

Objectif

Ajouter une fonctionnalité non disponible en librairie standard et illustrer la puissance du codesign matériel/logiciel.

Étapes réalisées

1) Conception du module PWM en VHDL
   - Compteur interne
   - Comparateur fréquence (freq)
   - Comparateur rapport cyclique (duty)
2) Création de l'interface Avalon
   - Registres accessibles par le processeur
   - Signaux writedata, readdata, address, chipselect
3) Intégration dans Platform Designer
4) Génération du système mis à jour
5) Pilotage du PWM via programme C

   Le processeur peut ainsi configurer dynamiquement :

   - La fréquence du signal
   - Le rapport cyclique
   - L'activation du module

Cette approche permet de décharger le processeur des calculs temps réel tout en conservant un contrôle logiciel flexible.

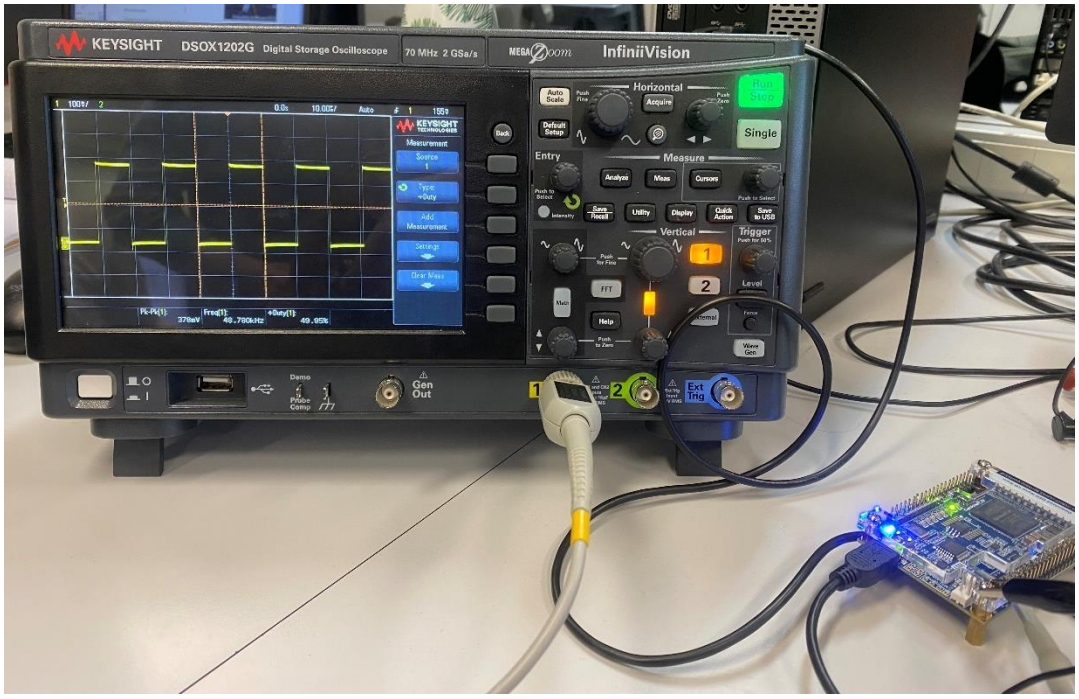



*Figure 12 - Photo qui visualise le PWM sous l'oscilloscope*

# IV. SPECIFICATIONS CIRCUIT ACQUISITION VITESSE VENT (ANEMOMETRE)

## 1. *Analyse fonctionnelle de la machine à états*

La machine à états a pour rôle de piloter la mesure de fréquence du signal issu de l'anémomètre sur une fenêtre de temps donnée, puis de valider la donnée mesurée.

Elle permet donc d'organiser le fonctionnement du système en trois phases successives : attente, mesure, puis validation.

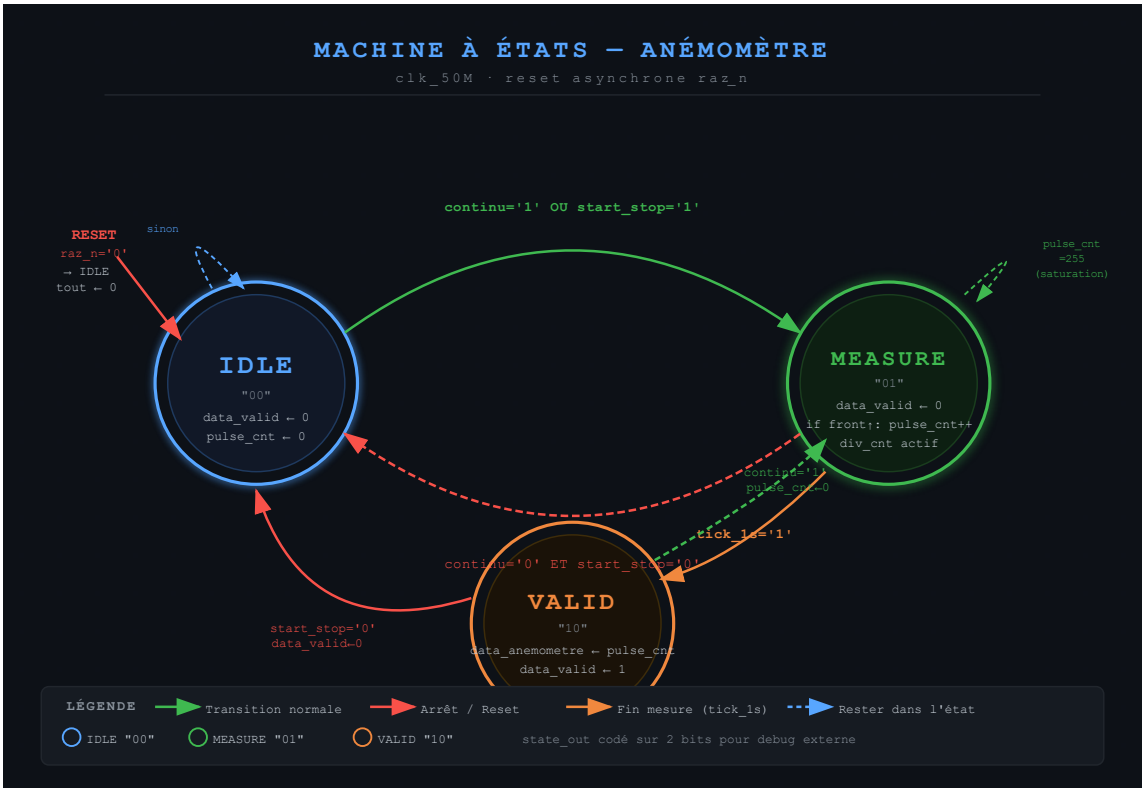



*Figure 13 - Illustration de la machine a état de l'anémomètre*

1. État IDLE

Cet état correspond à l'attente.

Dans cet état :

- Le système ne mesure pas,
- Le compteur d'impulsions est remis à zéro,
- Aucune donnée n'est déclarée valide (data_valid = 0)

2. État MEASURE

Cet état correspond à la phase de mesure.

Dans cet état :

- La fenêtre temporelle est lancée,
- Chaque front montant détecter sur le signal in_freq_anemometre incrémente le compteur pulse_cnt,
- Le système compte donc le nombre d'impulsions pendant la durée définie.

Deux cas sont possibles :

- Si la commande de mesure disparaît (continu = 0 et start_stop = 0), la machine retourne à IDLE,
- Si la durée de mesure est atteinte (tick_1s = 1 dans la version initiale), la machine passe à VALID.

3. État VALID

Cet état correspond à la validation du résultat.

Dans cet état :

- La valeur comptée est copiée vers la sortie data_anemometre,
- Le signal data_valid passe à 1 pour indiquer que la donnée est exploitable.

Ensuite :

- En mode continu, la machine repart automatiquement vers MEASURE pour effectuer une nouvelle mesure,
- En mode start_stop, elle attend que start_stop retombe à 0 avant de revenir à IDLE.

## 2. *Vérification de la simulation :*

Pour un premier test, la simulation montre que :

***state_out = 01 → MEASURE***

On observe que:

***data_anemometre ≈ 39 puis 40***

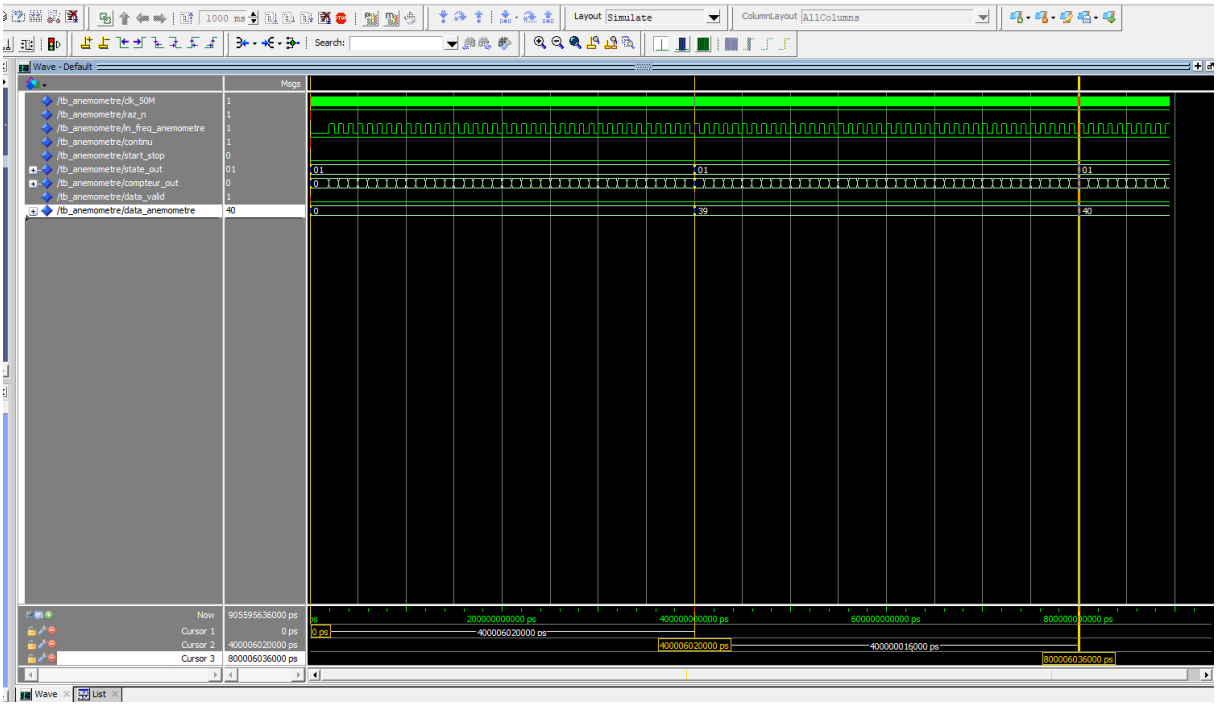



*Figure 14 - Simulation TB pour CLK_periode 8 ns et 900ms de simulation*

Ce résultat doit être analysé pour déterminer la fréquence réellement simulée.

La fenêtre de mesure du système est d'environ :

$$40\,000\,000\ ns\ =\ 40\ ms$$

Ainsi, pendant cette fenêtre, on compte **environ 40 impulsions**.

La fréquence correspondante est donc :

$$\boxed{f = \frac{40}{0.04} = 1000Hz}$$

Le signal d'entrée simulé correspond donc à **1 kHz**.

Pour simuler une fréquence de **100 Hz**, la période du signal doit être :

$$\boxed{T = \frac{1}{100} = 10\ ms}$$

Dans une fenêtre de **40 ms**, on doit alors obtenir :

$$\boxed{100\ \times 0.04 = 4\ \text{impulsions}}$$

Pour vérifier correctement le fonctionnement de l'anémomètre sur **une fenêtre de 1 seconde**, le testbench a été modifié afin de générer un signal d'entrée de **période 10 ms**.

Dans ces conditions, le système mesure environ :

***data_anemometre ≈ 99***

Ce qui correspond bien à la fréquence attendue de **100 Hz**, la différence de ±1 étant due à l'alignement temporel entre la fenêtre de mesure et les fronts du signal d'entrée.

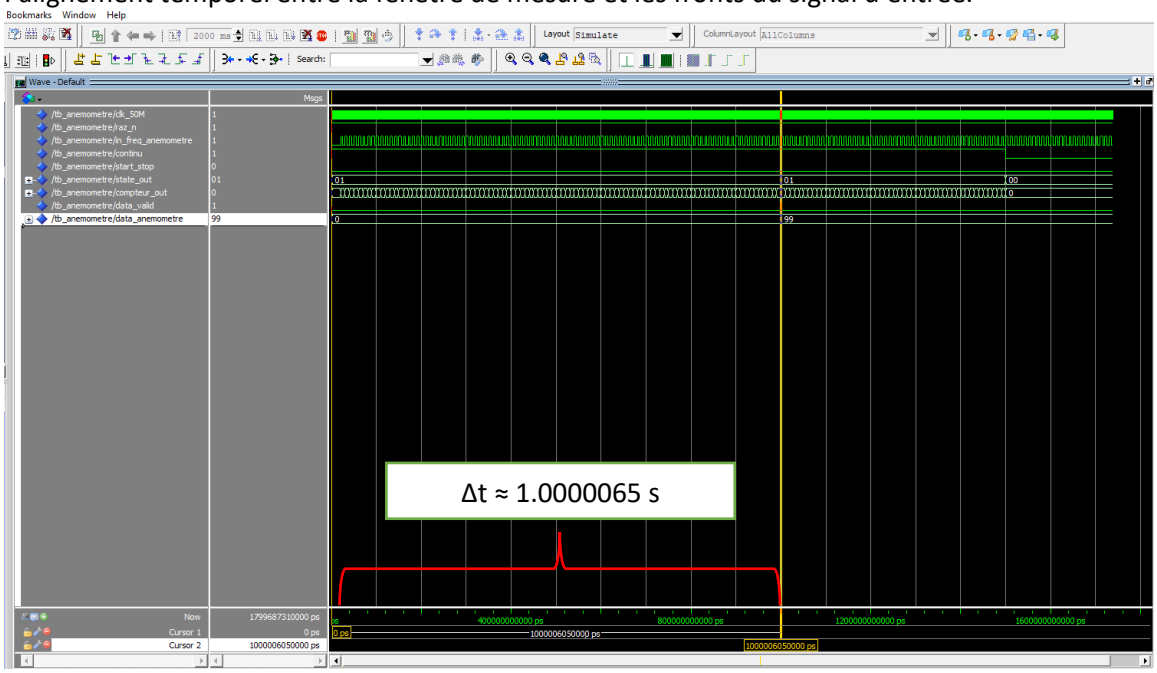



*Figure 15 - Simulation TB pour la mae de l'anémomètre*

## V. CONCLUSION

Ce projet de Codesign a permis de mettre en pratique l'ensemble des concepts fondamentaux de la conception numérique sur FPGA, depuis la description VHDL de composants simples jusqu'à l'intégration d'un système embarqué plus complet.

Les premiers exercices ont permis de consolider les bases essentielles de la conception logique :

- Conception de compteurs synchrones
- Gestion du reset et de la synchronisation des signaux
- Création et validation de testbenchs
- Simulation fonctionnelle sous ModelSim

Implémentation et validation expérimentale sur carte FPGA DE0-Nano

L'introduction d'un diviseur d'horloge a permis d'illustrer l'importance de l'adaptation des signaux temporels aux contraintes physiques réelles d'un système numérique.

La réalisation d'une machine à états finis (MAE) a ensuite permis d'approfondir la conception de systèmes séquentiels synchrones, en structurant le comportement du système autour d'états clairement définis et de transitions conditionnelles.

L'exercice sur le compteur avec anti-rebond de 20 ms a introduit une problématique réelle d'ingénierie : la gestion des signaux issus de boutons mécaniques. L'utilisation conjointe d'une machine à états et d'un timer a permis d'obtenir une solution robuste garantissant une seule incrémentation par appui, démontrant l'importance du filtrage temporel dans les systèmes numériques embarqués.

Le projet a également conduit à la conception d'un module d'anémomètre, basé sur une machine à états permettant de compter les impulsions issues d'un capteur de vitesse de vent pendant une fenêtre de mesure d'une seconde. La simulation a permis de vérifier le fonctionnement du système : pour un signal d'entrée de 100 Hz, le système mesure environ 99 impulsions sur une seconde, ce qui correspond au comportement attendu compte tenu des effets d'alignement temporel entre la fenêtre de mesure et les fronts du signal.

Enfin, la conception d'un SOPC via Platform Designer a constitué une étape importante du projet en intégrant un processeur softcore NIOS II. Cette partie a mis en évidence la complémentarité entre :

- Le développement matériel sous Quartus Prime
- Le développement logiciel embarqué
- L'intégration de composants propriétaires (ex : générateur PWM)

Ce projet démontre ainsi la capacité à concevoir, simuler, valider et intégrer un système numérique complet, en respectant une méthodologie d'ingénierie rigoureuse :

***Conception → Simulation → Implémentation → Validation expérimentale***

Au-delà des aspects techniques, ce travail renforce les compétences en :

- Structuration de projets FPGA
- Validation fonctionnelle par simulation
- Conception de machines à états
- Compréhension des architectures embarquées matériel / logiciel

Il constitue ainsi une base solide pour des projets FPGA plus avancés et pour l'ingénierie des systèmes embarqués à forte intégration.